

A Centralised End-to-End Test Framework for the AIB Microservices Estate

- **Author:** Muzammil
- **Status:** Proposal — for review by tech leads and architects
- **Version:** 1.0

Executive Summary

We propose a centralised, plug-and-play end-to-end (E2E) test framework that exercises full functional flows across our 100+ Spring Boot microservices estate. The framework spins up each microservice and its real dependencies — DB2, Oracle, Kafka, Schema Registry, Eureka, Config Server — as ephemeral containers via Testcontainers, then drives BDD-style scenarios that prove cross-service flows work end-to-end.

The framework is owned by a small platform/QE team and is consumed by service teams with effectively zero code change. Onboarding a new service to nightly E2E coverage is a one-line manifest entry plus a published Docker image. Tests are written in business-readable Gherkin so BAs and QE leads can review them.

A working proof of concept has been built and demonstrated. It runs a full producer → Kafka (Avro) → consumer → Oracle flow across two services with Eureka discovery, Spring Cloud Config Server, Feign clients, and circuit-breaker-backed fallbacks. The proof of concept also surfaced five real configuration issues during build-up — issues that would have produced production incidents — validating the central thesis: **this framework finds bugs that unit and contract tests miss.**

This document outlines the problem, the proposed solution, the trade-offs we deliberately took, and a phased rollout plan that won't disrupt service teams' delivery cadence.

1. Problem

1.1 Current state

Our estate has very strong unit test coverage — most services exceed 80% line coverage. Contract tests at the service boundary are also reasonable. What we **lack** is reliable, automated coverage of complete *functional flows* that span service boundaries:

"When upstream system X submits a securities update via REST, does it land on Kafka with the right Avro schema, get consumed by Service B, persist to Oracle with correct precision, and remain queryable downstream — every time, in every release?"

Today, the answer to that question comes from a mix of:

- Manual QA cycles in lower environments — slow, inconsistent, and a release-day bottleneck
- Production incidents that should have been caught earlier
- Each team's bespoke integration tests — usually skipped in CI, often broken, never reused across services

- Hope

1.2 What this framework is not

It is important to be precise about scope:

- **Not a unit-test replacement** — we already have strong unit coverage and aren't proposing to change that
- **Not a load or performance test framework** — that remains a separate concern (JMeter team, etc.)
- **Not a contract-test framework** — Pact and Spring Cloud Contract have their place; this framework operates at a different level
- **Not a UI/Selenium framework** — there's no UI in scope

It **is** a framework for testing functional flows that span at least two services, using their real images, with their real infrastructure dependencies, against assertions made directly on Kafka and the database (not via the services' own APIs, which would be circular).

1.3 The cost of not addressing this

For an estate of our size and regulatory profile, the cost compounds:

- Production incidents whose root causes were "configuration drift" or "schema mismatch"
- Engineering hours spent reproducing integration bugs locally because there's no shared environment that mirrors production reliably
- Release risk that grows with every new service and integration point
- Inability to refactor safely — nobody dares touch a service whose downstream impact is invisible

2. Approach

2.1 Core principles

The framework is designed around four principles that drive every implementation decision.

1. Tests are black-box. The framework treats each microservice as a deployed Docker image. It interacts only through public surfaces: REST endpoints, Kafka topics, database tables. It never reaches into a service's internals. This is what makes it portable across any service in the estate without modification.

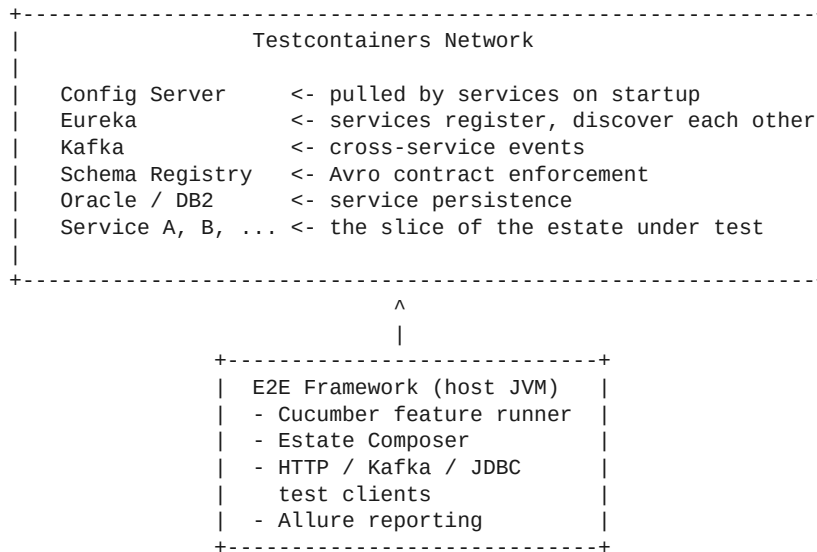
2. The framework is a product, not a project. It has its own repository, version, release cadence, and a small platform/QE team that owns it. Service teams are *consumers*, not contributors (in the main).

3. Tests describe business flows, not technical steps. A scenario reads "when a securities update is published, the consumer persists it to Oracle within 5 seconds with the correct enrichment rule" — not "POST to /securities, then poll Kafka, then SELECT from securities table." Business intent goes in the feature file; technical detail lives in reusable step libraries.

4. Real infrastructure, ephemeral lifecycle. Real Kafka, real Oracle, real DB2, real Schema Registry, real Eureka, real Config Server — every dependency runs as a container, fresh per Maven invocation. No mocked DB drivers. No in-memory Kafka stand-ins. The framework tests against the same infrastructure types that run in production.

2.2 Architecture at a glance

The framework composes a topology of containers per test invocation:



The Estate Composer is the heart of the framework. It reads a manifest, brings up the correct slice of services and infrastructure, wires them together, waits for them to be healthy, and exposes them to the scenario layer via test clients. Critically, scenarios specify which slice they need — a securities scenario does not require the entire 100-service estate to be running, only the services in its flow.

2.3 Onboarding a new service

Adding a service to nightly coverage is intentionally trivial:

1. Service team publishes a Docker image to the internal registry (most teams already do)
2. One line is added to the framework's `estate-manifest.yml` declaring the service
3. The service's existing `application.yml` is committed to a shared config repository the framework's Config Server reads from
4. Scenarios that need the service tag it (`@securities`, `@payments`, etc.) and the Estate Composer brings it up

That's the entire onboarding workflow. The service team writes no test code unless they choose to contribute scenarios for their domain.

2.4 Technology choices

Our stack and the rationale, deliberately opinionated:

Layer	Choice	Why
Test runner	JUnit 5	Industry standard, excellent IDE support
BDD layer	Cucumber-JVM (Gherkin)	BAs can review scenarios, step reuse across domains
HTTP assertions	REST Assured	De facto standard in the JVM world
Container orchestration	Testcontainers + custom Estate Composer	Per-test lifecycle, JVM-native config, no Docker Compose ceremony

Layer	Choice	Why
Schema Registry	Confluent (containerised)	Real schema validation, not mocks
Oracle	gvenzl/oracle -free:23-slim -faststart	Boots in ~30s on Linux, Oracle-compatible
DB2	icr.io/db2_community/db2	IBM's official community image
Async assertions	Awaitility	Robust polling for Kafka→DB lag
Reporting	Allure	Best Cucumber integration, audit-friendly
Build	Maven multi-module	Aligns with rest of estate

We deliberately did not choose:

- **Karate** — concise, but a separate DSL; Cucumber + Java keeps us in the JVM ecosystem the team already lives in
- **In-memory Kafka or H2 substitutes** — these don't catch the real bugs; if it's not real Kafka, you haven't tested Kafka
- **Docker Compose orchestration** — Testcontainers gives us per-test lifecycle, network isolation, and dynamic-port resolution that Compose makes painful

3. Evidence: A Working Proof of Concept

To validate the proposal, a working proof of concept has been built and is available for live demonstration. It exercises every layer of the AIB stack on a representative two-service flow:

```
POST /securities -> Service A (publisher)
|
+--> Feign call -> Service B (resolved through Eureka)
|   for enrichment-rule lookup (resilience-tested)
|
+--> Avro event -> Kafka topic securities.updates
                  (Schema Registry-validated)
|
+--> Service B consumer
      -> Oracle (Flyway-managed schema)
      -> idempotent persist with manual ack
```

Both services pull configuration from a containerised Spring Cloud Config Server. Both register with a containerised Eureka instance. Service A's Feign client is backed by a Resilience4j circuit breaker with a real fallback path. The framework asserts each checkpoint independently — HTTP response status, Avro event presence and field values on Kafka, persisted row in Oracle with correct decimal precision.

The proof of concept also surfaced five real configuration issues during build-up, each of which would otherwise have manifested as a production incident or hard-to-diagnose flaky behaviour:

1. The `-parameters` compiler flag, required by Spring Boot 3 for `@PathVariable` binding, is not set automatically when projects don't extend `spring-boot-starter-parent` — a silent 500-error class of bug

2. The `eureka.client.serviceUrl.defaultZone` property is a `Map<String, String>` whose key is camelCase; environment-variable binding strips that casing and fails silently
3. Eureka self-preservation triggers when heartbeats are tuned more aggressively than 30 seconds, causing the server to report DOWN even when registration is working — a documented Spring Cloud quirk
4. The Eureka client's healthcheck integration creates a startup race where transient DOWN states permanently lock services in DOWN, because heartbeats only renew leases and not status
5. The default `prefer-ip-address: true` on Windows + Docker Desktop + WSL2 picks the wrong network interface, registering services with unreachable IPs

Every one of these is a real bug, fixed in the proof of concept, with the fix documented inline in the configuration. **This is the framework doing its job before it has even shipped.**

4. Trade-offs and Open Questions

A proposal that hides its trade-offs is not credible. These are the deliberate decisions we made and the questions we'd like architecture to weigh in on.

4.1 Decisions taken

Centralised ownership, not federated. A single platform/QE team owns the framework and the central scenario suite. Service teams can contribute scenarios but the framework itself stays under one team's control. The alternative — every service team owns its slice — leads to fragmentation we've seen elsewhere in the estate.

Nightly cadence, not per-PR. The full E2E suite runs nightly, not on every pull request. A @smoke subset can run per-PR if a service team wants it, but the nightly is the gate. This keeps PR cycle times fast while still giving us coverage every 24 hours.

Real infrastructure containers, not mocks. Already justified above; calling it out as a deliberate cost — first run pulls ~3GB of images.

Black-box testing only. The framework deliberately does not have access to service internals. This is a feature: it means tests stay decoupled from implementation, and the framework is portable across services regardless of internal architecture.

4.2 Open questions for architecture

Things we'd like a steer on before Phase 2:

1. **Eureka and Config Server images for the test estate** — the proof of concept uses third-party Steeltoe images. For production rollout we recommend building AIB-internal Eureka and Config Server images hosted in our own registry, removing the third-party dependency. This needs a small platform investment to decide.
2. **Real third-party endpoints in nightly runs** — the framework currently assumes real calls to third parties (mainframe, payment gateways, etc.) using a controlled lower environment. Confirming this is acceptable for nightly automation, and how flakiness in those endpoints is triaged, needs an architecture decision.
3. **Test data lifecycle for shared schemas** — for services backed by shared Oracle/DB2 schemas (common in our estate), per-scenario data isolation needs a strategy. The framework supports several

approaches; choosing the default needs architectural input.

4. CI compute requirements — running 100+ services in nightly slices will need dedicated build agents with Docker access. We need confirmation that platform engineering can provide these, or we constrain the rollout accordingly.

5. Failure ownership and triage — when a nightly run fails on a securities flow at 2 AM, who is the on-the-hook triage owner? Framework team owns the framework, but not the bug. A clear RACI is needed before we make this a release gate.

5. Phased Rollout Plan

The rollout is deliberately staged so service teams are not asked to do anything until the framework is proven to work end-to-end on representative flows.

Phase 1 — Foundation (Weeks 1–6)

- Productionise the Estate Composer, step libraries, and reporting stack from the proof of concept
- Onboard one real AIB flow as the reference implementation (recommendation: securities-sync-secdb-to-collate — well-understood, exercises Kafka + DB2)
- Establish the framework's own repository, CI pipeline, and image-publishing workflow

Exit criterion: the reference flow runs green nightly for two consecutive weeks.

Phase 2 — Second and Third Verticals (Weeks 7–12)

- Onboard 2–3 more flows from different domains
- Document onboarding instructions for service teams
- Iterate on framework v2 based on the rough edges discovered during onboarding

Exit criterion: three independent flows running green nightly. Onboarding documentation reviewed and approved.

Phase 3 — Self-Service (Q2)

- Service teams begin onboarding their own services using the documented workflow
- Brown-bag sessions and consultation hours from the platform team
- Framework team focuses on reliability and reporting improvements

Exit criterion: ten flows under coverage. At least three onboarded by service teams without platform-team intervention.

Phase 4 — Quality Gate (Q3+)

- Passing the relevant nightly E2E suite becomes a release gate for promotion to UAT
 - Critical flows have circuit-breaker scenarios in addition to happy paths
 - Framework reaches steady-state operational maturity
-

6. Investment Required

This section is deliberately concrete.

People: A platform/QE team of 2–3 engineers, fully dedicated, for Phases 1 and 2 (3 months). Reduces to 1–2 engineers in steady-state operations from Phase 3 onwards.

Infrastructure:

- Dedicated build agents with Docker daemon access (estimate: 2 medium-spec VMs to start)
- Internal image registry capacity for the framework's own images and any internal Eureka/Config Server images
- Git repository for the framework
- Allure reporting dashboard hosting (low — can be a static-site bucket)

Time-to-first-value: approximately 6 weeks from kick-off to a single AIB flow running green nightly.

Time-to-broad-coverage: approximately 6 months to have 10+ flows under continuous coverage with self-service onboarding documented.

7. Risks

The honest list, with mitigations.

Risk	Likelihood	Mitigation
Build-agent Docker access blocked by infosec	Medium	Engage early with infosec; the proof of concept demonstrates the security profile; Testcontainers is widely accepted in regulated industries
Service teams resist adoption	Medium	Onboarding is one line; framework opt-out is to not declare your service; quality gate creates organic pull
Nightly run becomes flaky	Medium	Estate composition is deterministic; logs are captured per scenario; framework treats flakiness as a P1 framework bug not a "rerun and hope"
Third-party endpoints make tests flaky	High	WireMock recording fallback documented; @blocked-external tag for transparent failure
Test data pollution across runs	Low–Medium	Containers are ephemeral; named-volume hygiene enforced by framework
Framework team becomes a bottleneck	Medium	Phase 3 is explicitly self-service; documentation is a first-class deliverable, not an afterthought

8. Decision Asked For

We are asking the architecture forum to:

1. **Endorse the approach** described in §2 — central, ephemeral, black-box, BDD-driven
2. **Approve Phase 1 staffing** — 2–3 engineers for 6 weeks to deliver the reference flow

3. Take a position on the open questions in §4.2 — particularly internal Eureka/Config images, third-party-endpoint policy, and CI compute provisioning

4. Commit to reviewing Phase 1 exit criteria — to gate progression to Phase 2

A live demonstration of the proof of concept is available on request and is recommended before final approval.

End of proposal.